



K.N. Toosi University of Technology

OFFLINE JAVA GUIDE

Chapter 5

Inheritance & Collections

Creating class hierarchies with inheritance and method overriding, then replacing raw arrays with Java's powerful Collection framework — ArrayList, HashSet, HashMap, and more.

COURSE

Advanced Programming

SEMESTER

Spring 2026

DEPARTMENT

Computer Engineering

v1.0

Table of Contents

5.1	What is Inheritance?	Concept, parent & child
5.2	The extends Keyword	Syntax, field inheritance
5.3	Method Overriding	@Override, rules
5.4	The super Keyword	Access parent members
5.5	The Object Class	toString, equals, hashCode
5.6	instanceof & Type Casting	Checking & converting types
5.7	Why Collections?	Limitations of arrays
5.8	ArrayList	Dynamic lists, methods
5.9	HashSet	Unique elements, no duplicates
5.10	HashMap	Key-value pairs, methods
5.11	Iterating Over Collections	for-each, forEach, Iterator
5.12	Common Mistakes & Tips	Pitfalls every beginner hits

Inheritance lets a new class **acquire** the fields and methods of an existing class. The existing class is the **parent** (or superclass), and the new class is the **child** (or subclass).

Why Inheritance?

- › **Code reuse** — write common logic once in the parent, all children inherit it
- › **Logical hierarchy** — model real-world "is-a" relationships (a Car *is a* Vehicle)
- › **Extensibility** — add new types without modifying existing code

Key Terminology

TERM	MEANING
Superclass (parent)	The class being inherited from
Subclass (child)	The class that inherits
<code>extends</code>	The keyword that creates the inheritance relationship
" is-a " relationship	A Dog <i>is a</i> Animal — test: "A [child] is a [parent]" must make sense

NOTE

Java supports **single inheritance only** — a class can extend exactly **one** parent. A class cannot extend two classes at once. (But it can implement multiple interfaces — covered later.)

JAVA

```
1  public class Animal {
2      String name;
3      int age;
4
5      public Animal(String name, int age) {
6          this.name = name;
7          this.age = age;
8      }
9
10     public void eat() {
11         System.out.println(name + " is eating.");
12     }
13
14     public void sleep() {
15         System.out.println(name + " is sleeping.");
16     }
17 }
18
19 // Dog IS-A Animal - inherits name, age, eat(), sleep()
20 public class Dog extends Animal {
21     String breed;
22
23     public Dog(String name, int age, String breed) {
24         super(name, age); // call parent's constructor
25         this.breed = breed;
26     }
27
28     public void bark() {
29         System.out.println(name + " says: Woof!");
30     }
31 }
```

JAVA

```
1  Dog d = new Dog("Rex", 3, "German Shepherd");
2  d.eat(); // inherited from Animal
3  d.sleep(); // inherited from Animal
4  d.bark(); // Dog's own method
5  System.out.println(d.name); // inherited field
6  System.out.println(d.breed); // Dog's own field
```

Rex is eating. Rex is sleeping. Rex says: Woof! Rex German Shepherd

OUTPUT

What Gets Inherited

INHERITED?	WHAT
Yes	<code>public</code> and <code>protected</code> fields and methods
Yes	Default (package-private) fields and methods — only if in the same package
No	<code>private</code> fields and methods (not directly accessible)
No	Constructors (never inherited — must be called via <code>super()</code>)

Constructor Chain

When you create a `Dog`, Java **automatically calls** the parent constructor first. If the parent has no no-arg constructor, you **must** explicitly call `super(args)` as the first line:

```
1 public Dog(String name, int age, String breed) {
2     super(name, age); // MUST be first line - calls Animal(name, age)
3     this.breed = breed;
4 }
5
6 // If Animal had a no-arg constructor, Java would call it implicitly.
7 // But since Animal only has Animal(String, int), you MUST call super explicitly.
```

JAVA

MISTAKE

If the parent has **no** no-arg constructor and you forget `super(args)`, you get a compile error. The error message is often confusing — it won't say "missing super call", it will say "constructor X not found". Always check the parent's constructors.

Single Inheritance Only

```
1 // VALID
2 class Dog extends Animal { }
3 class Cat extends Animal { }
4
5 // INVALID - Java does not allow this
6 // class Dog extends Animal, Pet { } // compile error
```

JAVA

Overriding means the child class provides a **new implementation** of a method that already exists in the parent. The child method *replaces* the parent's version for objects of the child type.

```
1 public class Animal {
2     public void makeSound() {
3         System.out.println("Some generic animal sound");
4     }
5 }
6
7 public class Cat extends Animal {
8     @Override
9     public void makeSound() {
10        System.out.println("Meow!");
11    }
12 }
13
14 public class Dog extends Animal {
15     @Override
16     public void makeSound() {
17        System.out.println("Woof!");
18    }
19 }
```

JAVA

```
1 Animal a = new Animal();
2 Animal c = new Cat();
3 Animal d = new Dog();
4
5 a.makeSound(); // "Some generic animal sound"
6 c.makeSound(); // "Meow!"      - Cat's version runs
7 d.makeSound(); // "Woof!"      - Dog's version runs
```

JAVA

```
Some generic animal sound Meow! Woof!
```

OUTPUT

Overriding Rules

RULE	DETAIL
Same method signature	Same name, same parameters (exact match)
Same or narrower return type	Child can return a subtype (covariant return)
Same or less restrictive access	Cannot make an <code>public</code> method <code>private</code>
Cannot override <code>static</code> methods	Static methods are hidden, not overridden (different behavior)
Cannot override <code>final</code> methods	<code>final</code> prevents overriding by design
Cannot override <code>private</code> methods	Private methods are not visible to the child

Overriding vs Overloading

FEATURE	OVERRIDING	OVERLOADING
Where?	Parent and child class	Same class (or subclass)
Method name	Same	Same
Parameters	Must be identical	Must be different
Return type	Same or subtype	Can differ freely
Access modifier	Same or wider	No restriction
Binding	Runtime (dynamic)	Compile-time (static)
Annotation	<code>@Override</code>	None

TIP

Always use `@Override` when overriding. It catches typos and signature mismatches at compile time instead of silently creating a new overloaded method. In modern Java, it's considered a best practice to **always** use it.

`super` refers to the **parent class**. It has three uses — parallel to `this` :

1. Call Parent Constructor: `super(args)`

```
1  public class Vehicle {
2      String brand;
3      public Vehicle(String brand) { this.brand = brand; }
4  }
5
6  public class Car extends Vehicle {
7      int doors;
8      public Car(String brand, int doors) {
9          super(brand);    // calls Vehicle(String)
10         this.doors = doors;
11     }
12 }
```

JAVA

2. Access Parent's Field: `super.field`

```
1  public class Animal {
2      String type = "animal";
3  }
4
5  public class Dog extends Animal {
6      String type = "dog";
7
8      public void printTypes() {
9          System.out.println(this.type);    // "dog" - child's field
10         System.out.println(super.type);    // "animal" - parent's field
11     }
12 }
```

JAVA

3. Call Parent's Overridden Method: `super.method()`


```
1  public class Animal {
2      public void describe() {
3          System.out.println("I am an animal.");
4      }
5  }
6
7  public class Dog extends Animal {
8      @Override
9      public void describe() {
10         super.describe(); // call the parent's version first
11         System.out.println("I am a dog.");
12     }
13 }
14
15 new Dog().describe();
16 // "I am an animal."
17 // "I am a dog."
```

I am an animal. I am a dog.

OUTPUT

QUICK REFERENCE — THIS VS SUPER

- › `this.field` → child's field | `super.field` → parent's field
- › `this.method()` → child's method | `super.method()` → parent's method
- › `this(args)` → child's constructor | `super(args)` → parent's constructor
- › Both must be the **first line** in a constructor
- › Both cannot be used in a `static` context

Every class in Java **automatically inherits from** `java.lang.Object` — even if you don't write `extends Object`. This means every object has access to a set of universal methods.

Key Object Methods

METHOD	PURPOSE	COMMON OVERRIDE?
<code>toString()</code>	String representation of the object	Very common
<code>equals(Object)</code>	Compare two objects for content equality	Very common
<code>hashCode()</code>	Integer hash used by HashMap, HashSet	Common (with equals)
<code>getClass()</code>	Returns the runtime Class object	Rarely
<code>clone()</code>	Creates a copy of the object	Rare
<code>finalize()</code>	Called by garbage collector (deprecated)	Never

Default toString() — Not Useful

```
1 Student s = new Student("Ali", 20);
2 System.out.println(s);
3 // Default: Student@1b6d3586 - class name + hash, not useful
```

JAVA

Override toString() — The Fix

```
1 public class Student {
2     private String name;
3     private int age;
4
5     public Student(String name, int age) {
6         this.name = name;
7         this.age = age;
8     }
9
10    @Override
11    public String toString() {
12        return "Student{name='" + name + "', age=" + age + "}";
13    }
14 }
15
16 System.out.println(new Student("Ali", 20));
17 // Student{name='Ali', age=20}
```

JAVA

Student{name='Ali', age=20}

OUTPUT

Override equals()

JAVA

```
1  public class Student {
2      private String name;
3      private int id;
4
5      public Student(String name, int id) {
6          this.name = name;
7          this.id = id;
8      }
9
10     @Override
11     public boolean equals(Object obj) {
12         if (this == obj) return true;           // same reference
13         if (obj == null || getClass() != obj.getClass()) return false;
14         Student other = (Student) obj;         // safe to cast now
15         return id == other.id && name.equals(other.name);
16     }
17 }
```

WARNING

The `equals()` method must take `Object` as its parameter type — not your class type. Using `equals(Student other)` is **overloading**, not **overriding**. It won't be called by collections like `ArrayList.contains()` or `HashMap`.

TIP

Always override `toString()` for every class you create. It makes debugging vastly easier — you can simply print the object instead of writing custom print statements every time.

instanceof — Type Check

Tests whether an object **is an instance of** a particular class (or its subclass):

```
1  Animal a = new Dog("Rex", 3, "Shepherd");
2
3  System.out.println(a instanceof Animal); // true
4  System.out.println(a instanceof Dog);    // true  - Dog IS-A Animal
5  System.out.println(a instanceof Cat);    // false
6  System.out.println(a instanceof Object); // true  - everything is an Object
```

JAVA

```
true true false true
```

OUTPUT

Type Casting — Downcasting

A parent reference can point to a child object, but you need to **cast** to access child-specific methods:

```
1  Animal a = new Dog("Rex", 3, "Shepherd");
2
3  a.makeSound(); // OK - Animal has this method
4  // a.bark();   // COMPILER ERROR - Animal doesn't have bark()
5
6  // Downcast: tell the compiler "trust me, this is actually a Dog"
7  Dog d = (Dog) a;
8  d.bark();     // OK now - "Rex says: Woof!"
```

JAVA

WARNING

Unsafe downcasting causes a `ClassCastException` at runtime. Always check with `instanceof` first:

```
if (a instanceof Dog) { Dog d = (Dog) a; d.bark(); }
```

Upcasting — Always Safe

```
1  Dog d = new Dog("Rex", 3, "Shepherd");
2  Animal a = d; // upcast - always safe, no cast needed
3  a.makeSound(); // "Woof!" - Dog's overridden version runs
```

JAVA

Why Use Parent References?

```
1 // Works with ANY Animal - flexible code
2 public void makeAnimalSpeak(Animal a) {
3     a.makeSound(); // calls the correct version at runtime
4 }
5
6 makeAnimalSpeak(new Dog("Rex", 3, "Shepherd")); // "Woof!"
7 makeAnimalSpeak(new Cat("Mimi", 2));           // "Meow!"
```

Woof! Meow!

Array Limitations

Limitation	What it means
Fixed size	Cannot grow or shrink after creation
No built-in methods	No <code>.add()</code> , <code>.remove()</code> , <code>.contains()</code> , <code>.sort()</code>
Primitives only (directly)	Need <code>Integer[]</code> for objects — verbose
No key-value structure	Arrays are index-based only
No uniqueness enforcement	Arrays allow duplicate elements

What Collections Solve

- › **Dynamic size** — grow and shrink automatically
- › **Rich API** — add, remove, contains, sort, search, size, clear
- › **Type-safe with generics** — `ArrayList<String>` holds only Strings
- › **Specialized structures** — lists, sets, maps, queues, stacks

The Three Main Interfaces

Interface	Duplicates?	Order?	Access	Key Implementation
<code>List</code>	Yes	Yes (indexed)	By index	<code>ArrayList</code>
<code>Set</code>	No	No guarantee	By value	<code>HashSet</code>
<code>Map</code>	Keys: No	No guarantee	By key	<code>HashMap</code>

Import Statement

```
1 import java.util.ArrayList;
2 import java.util.HashSet;
3 import java.util.HashMap;
4
5 // Or import everything at once (common in learning):
6 import java.util.*;
```

JAVA

NOTE

Collections store **objects only**, not primitives. Java **autoboxes** primitives for you: `list.add(42)` becomes `list.add(Integer.valueOf(42))` automatically. This has a small performance cost but is usually negligible.

`ArrayList` is a **dynamic array** — it grows automatically when you add elements and shrinks when you remove them. It is the most commonly used collection in Java.

Creating an ArrayList

```
1 import java.util.ArrayList;
2
3 ArrayList<String> names = new ArrayList<>();
4 ArrayList<Integer> numbers = new ArrayList<>();
5 ArrayList<Double> scores = new ArrayList<>();
```

JAVA

NOTE

The `<>` (diamond operator) tells the compiler to infer the type from the left side. You can also write `new ArrayList<String>()` — the shorter form is standard since Java 7.

Essential Methods

```

1  ArrayList<String> list = new ArrayList<>();
2
3  // Add elements
4  list.add("Ali");
5  list.add("Sara");
6  list.add("Reza");
7  // ["Ali", "Sara", "Reza"]
8
9  // Add at specific index
10 list.add(1, "Mina");
11 // ["Ali", "Mina", "Sara", "Reza"]
12
13 // Access by index
14 System.out.println(list.get(0));    // "Ali"
15
16 // Size
17 System.out.println(list.size());    // 4
18
19 // Modify element at index
20 list.set(0, "Ali2");
21 // ["Ali2", "Mina", "Sara", "Reza"]
22
23 // Remove by index (returns removed element)
24 list.remove(0);
25 // ["Mina", "Sara", "Reza"]
26
27 // Remove by value (returns boolean)
28 list.remove("Reza");
29 // ["Mina", "Sara"]
30
31 // Check if contains
32 System.out.println(list.contains("Sara"));    // true
33
34 // Index of element
35 System.out.println(list.indexOf("Sara"));    // 1
36 System.out.println(list.indexOf("Ali"));    // -1 (not found)
37
38 // Check if empty
39 System.out.println(list.isEmpty());    // false
40
41 // Clear all
42 list.clear();
43 System.out.println(list.size());    // 0

```

Ali 4 Ali2 true 1 -1 false 0

OUTPUT

Convert to/from Array


```
1 ArrayList<String> list = new ArrayList<>();
2 list.add("A"); list.add("B"); list.add("C");
3
4 // ArrayList → Array
5 String[] arr = list.toArray(new String[0]);
6
7 // Array → ArrayList
8 String[] data = {"X", "Y", "Z"};
9 ArrayList<String> list2 = new ArrayList<>(Arrays.asList(data));
```

Iterating

```
1 ArrayList<String> list = new ArrayList<>();
2 list.add("Ali"); list.add("Sara"); list.add("Reza");
3
4 // for-each (preferred for reading)
5 for (String name : list) {
6     System.out.print(name + " ");
7 }
8
9 // Classic for loop (when you need the index)
10 for (int i = 0; i < list.size(); i++) {
11     System.out.println(i + ": " + list.get(i));
12 }
```

Ali Sara Reza

Complete Method Reference

A `HashSet` stores elements with **no duplicates**. It does not maintain insertion order. Use it when you need uniqueness and fast lookups.

Creating a HashSet

```
1 import java.util.HashSet;
2
3 HashSet<String> names = new HashSet<>();
4 HashSet<Integer> numbers = new HashSet<>();
```

JAVA

Duplicates Are Automatically Ignored

```
1 HashSet<String> set = new HashSet<>();
2 set.add("Ali");
3 set.add("Sara");
4 set.add("Ali");    // duplicate - ignored
5 set.add("Reza");
6
7 System.out.println(set.size()); // 3 - "Ali" was not added twice
```

JAVA

3

OUTPUT

Essential Methods

```
1 HashSet<String> set = new HashSet<>();
2 set.add("Ali"); set.add("Sara"); set.add("Reza");
3
4 // Contains
5 System.out.println(set.contains("Sara")); // true
6 System.out.println(set.contains("Mina")); // false
7
8 // Remove
9 set.remove("Ali");
10 // ["Sara", "Reza"] (order not guaranteed)
11
12 // Size & empty
13 System.out.println(set.size()); // 2
14 System.out.println(set.isEmpty()); // false
15
16 // Clear
17 set.clear();
18 System.out.println(set.size()); // 0
19
20 // Iterate (no index access - order not guaranteed)
21 for (String name : set) {
22     System.out.print(name + " ");
23 }
```

JAVA

When to Use Set vs List

USE LIST WHEN	USE SET WHEN
Duplicates are allowed	Duplicates must be prevented
Order matters	Order doesn't matter
Access by index needed	Only need contains/remove/add
Same element can appear multiple times	Each element appears at most once

Common Use Case — Remove Duplicates from a List

```
1 ArrayList<String> list = new ArrayList<>();
2 list.add("Ali"); list.add("Sara"); list.add("Ali"); list.add("Reza");
3
4 HashSet<String> unique = new HashSet<>(list);
5 System.out.println(unique); // [Ali, Sara, Reza] - no duplicates
```

JAVA

NOTE

If you need **ordered uniqueness**, use `LinkedHashSet` (keeps insertion order) or `TreeSet` (sorted order). Both implement `Set` and have the same API as `HashSet`.

A `HashMap` stores **key-value pairs**. Each key maps to exactly one value. Keys must be unique — adding a duplicate key overwrites the previous value.

Creating a HashMap

```
1 import java.util.HashMap;
2
3 HashMap<String, Integer> ages = new HashMap<>();
4 HashMap<String, String> emails = new HashMap<>();
```

JAVA

Essential Methods

```

1  HashMap<String, Integer> map = new HashMap<>();
2
3  // Put (add/update)
4  map.put("Ali", 20);
5  map.put("Sara", 22);
6  map.put("Reza", 21);
7  map.put("Ali", 23);    // overwrites Ali's age – keys are unique
8
9  // Get by key
10 System.out.println(map.get("Sara"));    // 22
11 System.out.println(map.get("Mina"));    // null (not found)
12
13 // Check if key exists
14 System.out.println(map.containsKey("Ali"));    // true
15 System.out.println(map.containsKey("Mina"));    // false
16
17 // Check if value exists
18 System.out.println(map.containsValue(22));    // true
19
20 // Remove by key
21 map.remove("Reza");
22
23 // Size
24 System.out.println(map.size());    // 2
25
26 // Iterate over entries (key-value pairs)
27 for (Map.Entry<String, Integer> entry : map.entrySet()) {
28     System.out.println(entry.getKey() + " → " + entry.getValue());
29 }
30
31 // Iterate over keys only
32 for (String key : map.keySet()) {
33     System.out.println(key);
34 }
35
36 // Iterate over values only
37 for (Integer val : map.values()) {
38     System.out.println(val);
39 }
40
41 // Clear
42 map.clear();

```

```

22 null true false true 2 Ali → 23 Sara → 22 Ali Sara 23 22

```

OUTPUT

getOrDefault() — Safe Retrieval

```

1  HashMap<String, Integer> scores = new HashMap<>();
2  scores.put("Math", 90);
3
4  // Returns value if key exists, otherwise the default
5  int math = scores.getOrDefault("Math", 0);    // 90
6  int phys = scores.getOrDefault("Physics", 0); // 0 (not found, returns default)

```

putIfAbsent() — Add Only if Missing

JAVA

```
1  HashMap<String, Integer> map = new HashMap<>();
2  map.put("Ali", 20);
3
4  map.putIfAbsent("Ali", 25); // ignored - "Ali" already exists
5  map.putIfAbsent("Sara", 22); // added - "Sara" doesn't exist
6  System.out.println(map.get("Ali")); // 20 (not overwritten)
```

Common Use Cases

SCENARIO	KEY	VALUE
Phone book	Name (<code>String</code>)	Phone number (<code>String</code>)
Word frequency	Word (<code>String</code>)	Count (<code>Integer</code>)
Student grades	Student ID (<code>String</code>)	GPA (<code>Double</code>)
Configuration	Setting name (<code>String</code>)	Setting value (<code>String</code>)

MISTAKE

Do not use `HashMap<int, String>` — primitives cannot be generic type arguments. Use `HashMap<Integer, String>`. Autoboxing handles the conversion automatically.

for-each Loop (Simplest)

```
1 // List
2 ArrayList<String> list = new ArrayList<>();
3 list.add("A"); list.add("B"); list.add("C");
4 for (String s : list) { System.out.print(s + " "); }
5 // A B C
6
7 // Set
8 HashSet<String> set = new HashSet<>(list);
9 for (String s : set) { System.out.print(s + " "); }
10 // A B C (order may vary)
11
12 // Map entries
13 HashMap<String, Integer> map = new HashMap<>();
14 map.put("x", 1); map.put("y", 2);
15 for (Map.Entry<String, Integer> e : map.entrySet()) {
16     System.out.println(e.getKey() + "=" + e.getValue());
17 }
```

JAVA

forEach() with Lambda (Java 8+)

```
1 ArrayList<String> list = new ArrayList<>();
2 list.add("Ali"); list.add("Sara"); list.add("Reza");
3
4 list.forEach(name → System.out.println(name));
5
6 HashMap<String, Integer> map = new HashMap<>();
7 map.put("Ali", 20); map.put("Sara", 22);
8
9 map.forEach((key, val) → System.out.println(key + ": " + val));
```

JAVA

Iterator — When You Need to Remove While Iterating

```
1 ArrayList<String> list = new ArrayList<>();
2 list.add("A"); list.add("B"); list.add("C"); list.add("D");
3
4 // Remove elements while iterating – safe with Iterator
5 Iterator<String> it = list.iterator();
6 while (it.hasNext()) {
7     String s = it.next();
8     if (s.equals("B") || s.equals("D")) {
9         it.remove(); // safe – uses the iterator's own remove method
10    }
11 }
12 System.out.println(list); // [A, C]
```

JAVA

Do **not** call `list.remove()` inside a for-each loop — it throws `ConcurrentModificationException`. Use an explicit `Iterator` with `it.remove()`, or use `list.removeIf()` (Java 8+):

```
list.removeIf(s → s.equals("B") || s.equals("D"));
```

removeIf() — Filter by Condition (Java 8+)

```
1  ArrayList<Integer> nums = new ArrayList<>();
2  nums.add(1); nums.add(2); nums.add(3); nums.add(4); nums.add(5);
3
4  // Remove all even numbers
5  nums.removeIf(n → n % 2 == 0);
6  System.out.println(nums); // [1, 3, 5]
```

JAVA

sort() — Sort a List

```
1  ArrayList<String> names = new ArrayList<>();
2  names.add("Reza"); names.add("Ali"); names.add("Sara");
3
4  Collections.sort(names);
5  System.out.println(names); // [Ali, Reza, Sara]
6
7  ArrayList<Integer> nums = new ArrayList<>();
8  nums.add(5); nums.add(1); nums.add(3);
9  Collections.sort(nums);
10 System.out.println(nums); // [1, 3, 5]
```

JAVA

QUICK REFERENCE — WHICH ITERATION METHOD?

- › Just reading → `for-each` loop (simplest)
- › Need index → classic `for (int i = 0; i < list.size(); i++)`
- › Need to remove during iteration → `Iterator` or `removeIf()`
- › Filter by condition → `removeIf()`
- › One-liner action → `forEach()` with lambda

1. Forgetting `super()` in Constructor

```
1  class Dog extends Animal {
2      public Dog(String name) {
3          // If Animal has no no-arg constructor, this won't compile
4          // Fix: add super(name); as the first line
5      }
6  }
```

JAVA

2. Wrong Parameter Type in `equals()`

```
1  // WRONG - this is overloading, not overriding
2  public boolean equals(Student other) { ... }
3
4  // CORRECT - must take Object to override
5  @Override
6  public boolean equals(Object obj) { ... }
```

JAVA

3. Using Primitives as Generic Types

```
1  // WRONG
2  ArrayList<int> list = new ArrayList<>();
3
4  // CORRECT
5  ArrayList<Integer> list = new ArrayList<>();
```

JAVA

4. `ConcurrentModificationException`

```
1  ArrayList<String> list = new ArrayList<>();
2  list.add("A"); list.add("B"); list.add("C");
3
4  // WRONG - throws ConcurrentModificationException
5  for (String s : list) {
6      if (s.equals("B")) list.remove(s);
7  }
8
9  // CORRECT - use Iterator or removeIf
10 list.removeIf(s → s.equals("B"));
```

JAVA

5. Unsafe Downcasting Without `instanceof`

```

1  Animal a = new Cat("Mimi", 2);
2
3  // WRONG - ClassCastException if wrong type
4  Dog d = (Dog) a;
5
6  // CORRECT - check first
7  if (a instanceof Dog) {
8      Dog d = (Dog) a;
9      d.bark();
10 }

```

6. HashMap get() Returns null — Not an Error

```

1  HashMap<String, Integer> map = new HashMap<>();
2  int val = map.get("missing");
3
4  // If val is used as int, this is fine (val = 0)
5  // But if used as Integer, val is null - calling val.intValue() would crash
6  // Safe pattern:
7  int val = map.getOrDefault("missing", 0);

```

7. Confusing List.add() and List.set()

```

1  ArrayList<String> list = new ArrayList<>();
2  list.add("A"); list.add("B"); // ["A", "B"]
3
4  list.add(0, "C"); // INSERT at index 0 → ["C", "A", "B"]
5  list.set(0, "D"); // REPLACE at index 0 → ["D", "A", "B"]

```

8. HashSet Order Is Unpredictable

```

1  HashSet<String> set = new HashSet<>();
2  set.add("C"); set.add("A"); set.add("B");
3  System.out.println(set);
4  // Might print [A, B, C] or [C, A, B] - order is NOT guaranteed
5  // If order matters, use LinkedHashSet instead

```

QUICK REFERENCE — COLLECTION SELECTION GUIDE

- › Need ordered elements with duplicates → `ArrayList`
- › Need unique elements, order doesn't matter → `HashSet`
- › Need unique elements in insertion order → `LinkedHashSet`
- › Need sorted unique elements → `TreeSet`
- › Need key-value lookups → `HashMap`
- › Need sorted key-value pairs → `TreeMap`
- › Need fixed size with primitives → plain **array** (still valid!)
- › Need thread safety → `CopyOnWriteArrayList` , `ConcurrentHashMap` (advanced)

This document was compiled from the following resources. All content was adapted and restructured for educational use.

[1] **Programiz — Java Inheritance**

extends keyword, method overriding, super keyword, Object class

<https://www.programiz.com/java-programming/inheritance>

[2] **Programiz — Java Collections**

ArrayList, HashSet, HashMap tutorials with examples

<https://www.programiz.com/java-programming/collections>

[3] **GeeksforGeeks — Inheritance in Java**

Inheritance rules, method overriding vs overloading, instanceof

<https://www.geeksforgeeks.org/inheritance-in-java/>

[4] **GeeksforGeeks — Java Collections Framework**

List, Set, Map interfaces and implementations, iteration patterns

<https://www.geeksforgeeks.org/collections-in-java/>

[5] **Oracle — Java Tutorials — Collections**

Interface hierarchy, usage guidelines, best practices

<https://docs.oracle.com/javase/tutorial/collections/>

CREATED BY

Sepehr Ghardashi

sepehr.ghardashi@gmail.com